

GUIA DE ESTUDO

IESPFLIX

Projeto Flix Backend

Spring Boot, requisitos obrigatórios, código atual e roteiro para apresentação

Versão estudada

Guia reconstruído a partir do código atual validado em 01/06/2026. A suíte executada possui 55 testes sem falhas. Este material substitui os guias antigos.

Aluno(a): Samantha Hellen

Disciplina: Tecnologias para Backend

Projeto: API REST de streaming IESPFLIX

Professor: Rodrigo Fujioka

Data de atualização: 01/06/2026

Como usar este guia

Este documento foi escrito para duas situações: estudar o projeto do zero e revisar rapidamente antes da apresentação. Leia primeiro as Partes I e II para entender Spring Boot e a organização do código. Depois estude a Parte III, que liga cada requisito obrigatório aos arquivos atuais. Na véspera da apresentação, use as Partes V e VI como roteiro prático.

Resumo em uma frase

O IESPFLIX é uma API REST em Spring Boot para cadastro de usuários, autenticação, catálogo de filmes e séries, favoritos e cartões tokenizados, com persistência H2, validações, Swagger, Spring Security e integração externa ViaCEP.

Sumário

- ☐ Parte I - Fundamentos: API REST, Spring Boot, Maven e injeção de dependência
- ☐ Parte II - Anatomia do projeto atual: pacotes, banco, DTOs, validações e fluxo
- ☐ Parte III - Requisitos obrigatórios: RF1, RF3, RF4, RF7, RF8, RF10, RF11, RNF1 e RNF2
- ☐ Parte IV - Tópicos avaliados: JPQL, padrões, tratamento de erros, ViaCEP, testes e JaCoCo
- ☐ Parte V - Demonstração no Swagger e inspeção pelo H2
- ☐ Parte VI - Roteiro oral, perguntas prováveis, checklist e glossário

O que mudou em relação aos guias antigos

Antes	Agora no código atual	Motivo
Java 21	Java 17	Compatibilidade com o ambiente instalado e compilação reproduzível.
Cadastro recebia UsuarioDTO	Cadastro recebe CadastroUsuarioDTO	RF1 exige confirmação de senha e dados do cartão no mesmo cadastro.
Cartão explicado como CRUD simples	Número completo e CVV não são persistidos	Reduz exposição de dados sensíveis; ficam token, bandeira e últimos 4 dígitos.
Admin descrito sem Swagger visual	OpenApiConfig cria basicAuth e botão Authorize	Facilita demonstrar RF11 pelo Swagger.
Tipo de conteúdo aberto	ConteudoDTO aceita somente FILME ou SERIE	Evita registros inválidos como STRING.
Filme era o foco	Conteudo é o catálogo principal	Um único modelo representa filmes e séries e permite separar favoritos.

Parte I - Fundamentos de Spring Boot

1. O que é backend?

Backend é a parte do sistema executada no servidor. Ele recebe requisições, aplica regras de negócio, consulta ou altera dados e devolve respostas. No IESPFLIX, o navegador ou Swagger envia JSON para a API, e o backend responde com JSON e um código HTTP.

2. O que é uma API REST?

API é uma interface para comunicação entre sistemas. REST é um estilo arquitetural que organiza recursos por URLs e usa os verbos do HTTP. No projeto, usuários, conteúdos, favoritos e cartões são recursos acessíveis por endpoints.

Verbo	Uso comum	Exemplo no projeto	Resposta típica
GET	Consultar	GET /api/v1/conteudos	200 OK
POST	Criar	POST /api/v1/usuarios	201 Created
PUT	Atualizar por completo	PUT /api/v1/metodos-pagamento/{id}	200 OK
PATCH	Alterar parcialmente	PATCH /api/v1/assinaturas/{id}/cancelar	200 OK
DELETE	Excluir	DELETE /api/v1/conteudos/{id}	204 No Content

3. Códigos HTTP que você deve saber

Código	Significado	Quando aparece
200	OK	Consulta, login ou atualização concluída.
201	Created	Cadastro criado, como usuário, conteúdo ou favorito.
204	No Content	Exclusão concluída sem corpo na resposta.
400	Bad Request	JSON inválido ou falha de Bean Validation.
401	Unauthorized	Credenciais ausentes ou incorretas.
403	Forbidden	Usuário autenticado, mas sem permissão de administrador.
404	Not Found	Recurso não encontrado.
409	Conflict	E-mail, CPF/CNPJ ou favorito duplicado.
500	Internal Server Error	Falha inesperada não tratada especificamente.

4. O que é Spring Boot?

Spring Boot é um framework Java que facilita criar aplicações web. Ele configura automaticamente o servidor, integra bibliotecas e encontra classes anotadas. Em vez de montar toda a infraestrutura manualmente, você declara intenções com anotações.

Anotação	O que informa ao Spring	Onde aparece
@SpringBootApplication	Classe principal e configuração automática.	TechbackApplication.java
@RestController	Classe que recebe requisições HTTP e devolve	Controllers

Anotação	O que informa ao Spring	Onde aparece
	JSON.	
@RequestMapping	Prefixo da URL de um controller.	Ex.: /api/v1/conteudos
@Service	Classe com regra de negócio.	Services
@Repository	Componente de acesso ao banco.	Repositories
@Entity	Classe persistida como tabela.	Models
@Configuration	Classe que fornece configurações e beans.	config
@Bean	Objeto criado e gerenciado pelo Spring.	SecurityConfig, ModelMapperConfig

5. Classe principal e inicialização

A aplicação começa em `src/main/java/br/uniesp/si/techback/TechbackApplication.java`.

```
@SpringBootApplication
@EnableFeignClients
public class TechbackApplication {
    public static void main(String[] args) {
        SpringApplication.run(TechbackApplication.class, args);
    }
}
```

@SpringBootApplication inicia o projeto e procura componentes nos subpacotes. @EnableFeignClients habilita interfaces declarativas para chamar APIs externas, como o ViaCEP.

6. Maven e pom.xml

O Maven baixa bibliotecas, compila e executa testes. O arquivo `pom.xml` é a lista de dependências e plugins do projeto. A propriedade `java.version` está configurada como 17.

Dependência ou plugin	Função no projeto
spring-boot-starter-web	API REST, JSON e servidor web embutido.
spring-boot-starter-data-jpa	Persistência com JPA e Hibernate.
h2	Banco local leve para desenvolvimento.
spring-boot-starter-validation	Bean Validation: @NotBlank, @Pattern, @Email etc.
spring-boot-starter-security	HTTP Basic, autorização por perfil e BCrypt.
spring-cloud-starter-openfeign	Cliente HTTP declarativo para ViaCEP.
springdoc-openapi-starter-webmvc-ui	Documentação Swagger UI.
flyway-core	Suporte a migrações; desativado no perfil dev atual.
lombok	Gera getters, setters, builders e construtores.
spring-boot-starter-test	JUnit, Mockito, MockMvc e ferramentas de teste.
jacoco-maven-plugin	Relatório de cobertura em <code>target/site/jacoco/index.html</code> .

Comandos essenciais

Iniciar: `.\mvnw.cmd spring-boot:run` | Testar: `.\mvnw.cmd test` | Swagger:
<http://localhost:8080/swagger-ui/index.html>

Parte II - Anatomia do projeto atual

7. Arquitetura em camadas

O projeto aplica separação de responsabilidades. Cada camada resolve um tipo de problema. Essa organização facilita manutenção, teste e explicação durante a apresentação.

Camada	Responsabilidade	Exemplo atual
Controller	Recebe HTTP, lê parâmetros e chama service.	UsuarioController, ConteudoController
DTO	Define dados de entrada e saída da API.	CadastroUsuarioDTO, ConteudoDTO
Validation	Bloqueia dados inválidos antes da regra de negócio.	CpfCnpjValidator, SenhaForteValidator
Service	Aplica regras e coordena operações.	UsuarioService, MetodoPagamentoService
Repository	Acessa o banco com Spring Data JPA.	UsuarioRepository, ConteudoRepository
Model / Entity	Representa tabelas do banco.	Usuario, Conteudo, Favorito
Config	Define segurança, Swagger e beans.	SecurityConfig, OpenApiConfig
Exception	Transforma erros em respostas padronizadas.	GlobalExceptionHandler

8. Fluxo completo de uma requisição

1. O Swagger envia uma requisição HTTP com JSON.
2. O Controller recebe a requisição e usa @Valid no DTO.
3. O Bean Validation verifica formato, obrigatoriedade e validadores customizados.
4. O Service aplica regras de negócio, como duplicidade, autorização e hash.
5. O Repository consulta ou salva a Entity no H2.
6. O Service monta um DTO de resposta sem expor segredos.
7. O Controller devolve JSON e um código HTTP.
8. Se houver erro, o GlobalExceptionHandler padroniza a resposta.

Fluxo resumido

Swagger -> Controller -> DTO + Validation -> Service -> Repository -> H2 -> DTO de resposta -> JSON

9. Pacotes do núcleo IESPFLIX

Pacote	Classes principais	Por que estudar
controller	UsuarioController, AuthController, ConteudoController, FavoritoController, OutrosControllers	Endpoints centrais da demonstração.
service	UsuarioService, AuthService, ConteudoService, FavoritoService, MetodoPagamentoService	Regras obrigatórias.
repository	UsuarioRepository, ConteudoRepository, FavoritoRepository, MetodoPagamentoRepository	Persistência e JPQL.
model	Usuario, MetodoPagamento, Conteudo, Favorito, Plano, Assinatura	Tabelas do domínio.

Pacote	Classes principais	Por que estudar
dto	CadastroUsuarioDTO, AtualizacaoCartaoDTO, ConteudoDTO, LoginRequestDTO	Contrato JSON e validações.
config	SecurityConfig, OpenApiConfig	Autorização e botão Authorize.
validation	CpfCnpj, SenhaForte e validadores	Bean Validation customizada.

10. Módulos didáticos legados

O repositório também contém módulos criados anteriormente para praticar CRUD, mapper, JPQL e serviço externo: Filme, Produto, Cliente, Categoria, Pedido e Funcionario. Eles continuam úteis, mas não são o catálogo principal da apresentação.

Não confunda

Para RF8, RF10 e RF11 use Conteudo e as rotas /api/v1/conteudos. A entidade Filme e a rota /filmes são exemplos antigos preservados no projeto.

11. JPA, Hibernate e entidades

JPA é uma especificação Java para persistência. Hibernate é a implementação usada pelo Spring Boot. Uma classe com @Entity vira uma tabela. @Id indica chave primária e @GeneratedValue deixa o banco gerar o ID.

Entidade	Tabela	Campos importantes	Uso
Usuario	usuarios	email, senhaHash, cpfCnpj, perfil	Cadastro, login e autorização.
MetodoPagamento	metodo_pagamento	ultimos4, bandeira, tokenGateway	Cartão sem armazenar número completo ou CVV.
Conteudo	conteudo	titulo, tipo, ano, duração, relevância, sinopse, trailerUrl, gênero	Catálogo de filmes e séries.
Favorito	favoritos	usuarioid, conteudoid	Relaciona usuário ao conteúdo favorito.
Plano	plano	codigo, limiteDiario, preco	Bonificação: tipos de plano.
Assinatura	assinatura	usuarioid, planoid, status, datas	Bonificação: vínculo do usuário com plano.

```
@Entity
@Table(name = "conteudo")
public class Conteudo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    private String tipo; // FILME ou SERIE
    private Integer ano;
    private Integer duracaoMinutos;
    private BigDecimal relevancia;
    private String sinopse;
    private String trailerUrl;
    private String genero;
}
```

12. DTOs: contrato da API

DTO significa Data Transfer Object. DTOs controlam o que entra e sai da API. Eles evitam expor a estrutura inteira da entidade e são o lugar ideal para Bean Validation.

DTO	Uso
CadastroUsuarioDTO	Entrada completa do RF1: dados pessoais, senha confirmada e cartão.
UsuarioDTO	Resposta e edição de usuário sem devolver senha hash.
LoginRequestDTO / LoginResponseDTO	Entrada e saída da autenticação.
AtualizacaoCartaoDTO	Entrada do RF7 com número, validade, CVV e titular.
MetodoPagamentoDTO	Representação persistida e operações adicionais de cartão.
ConteudoDTO	Contrato do catálogo com validação de FILME ou SERIE.
FavoritoDTO	Relaciona usuariold e conteudold.

Segurança de saída

@JsonProperty(access = WRITE_ONLY) permite receber dados sensíveis no JSON de entrada, mas impede que eles apareçam na resposta. O projeto aplica isso à senha, confirmação de senha, número do cartão, CVV, validade recebida e token do gateway.

13. Bean Validation

Bean Validation bloqueia entradas erradas antes de o service executar. O Controller ativa isso com @Valid. O projeto combina anotações prontas e validadores próprios.

Anotação	Validação	Exemplo
@NotBlank	Texto obrigatório e não vazio.	nomeCompleto, email, senha
@NotNull	Valor obrigatório.	dataNascimento, ano, duração
@Email	Formato básico de e-mail.	CadastroUsuarioDTO.email
@Past	Data deve estar no passado.	dataNascimento
@Pattern	Texto precisa casar com regex.	cartão, validade, CVV, tipo
@Min / @Max	Intervalo numérico.	ano, duração, relevância
@CpfCnpj	Dígitos verificadores de CPF ou CNPJ.	cpfCnpj
@SenhaForte	8+ caracteres, maiúscula, minúscula, número e especial.	senha

```
@NotBlank(message = "Tipo é obrigatório (FILME ou SERIE)")
@Pattern(regex = "(?i)FILME|SERIE",
        message = "Tipo deve ser FILME ou SERIE")
private String tipo;
```

14. H2 e persistência

O perfil dev usa H2 em arquivo. Isso significa que os dados continuam salvos após fechar e abrir o projeto. O perfil test usa H2 em memória e create-drop, portanto os testes não sujam o banco usado na apresentação.

Ambiente	URL JDBC	DDL auto	Comportamento
dev	jdbc:h2:file:~/teckback20262	update	Mantém dados entre reinícios.
test	jdbc:h2:mem:testdb	create-drop	Cria banco temporário e apaga ao terminar.

Console H2: <http://localhost:8080/h2>

```
JDBC URL: jdbc:h2:file:~/teckback20262
User Name: sa
Password: [deixe vazio]
```

Parte III - Requisitos do professor no código atual

15. Mapa geral dos requisitos obrigatórios

ID	Pedido do professor	Implementação atual	Endpoint principal
RF1	Cadastrar usuário com dados pessoais, senha confirmada e cartão.	CadastroUsuarioDTO + UsuarioService + MetodoPagamentoService	POST /api/v1/usuarios
RF3	Salvar senha com HASH.	BCryptPasswordEncoder e campo senhaHash	POST /api/v1/usuarios
RF4	API de login.	AuthController + AuthService	POST /api/v1/auth/login
RF7	Alterar cartão após autenticação.	AtualizacaoCartaoDTO + HTTP Basic + autorização por dono	PUT /api/v1/metodos-pagamento/{id}
RF8	Detalhar filmes e séries.	Conteudo + ConteudoDTO	GET /api/v1/conteudos/{id}
RF10	Listar favoritos por tipo.	FavoritoRepository filtra FILME ou SERIE	GET .../filmes e GET .../series
RF11	Exigir admin para cadastrar filmes e séries.	SecurityConfig hasRole(ADMIN)	POST /api/v1/conteudos
RNF1	Versionamento Git.	Repositório Git com remote origin.	GitHub do projeto
RNF2	README executável.	README.md com execução, Swagger, H2 e fluxos.	README.md

16. RF1 - Cadastro completo do usuário

O cadastro público fica em POST /api/v1/usuarios. O Controller recebe CadastroUsuarioDTO, não UsuarioDTO. Essa separação existe porque o cadastro inicial precisa de campos extras: confirmarSenha e os dados do cartão.

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public UsuarioDTO criar(@Valid @RequestBody CadastroUsuarioDTO dto) {
    return usuarioService.criar(dto);
}
```

Dentro de UsuarioService.criar, a ordem principal é:

9. Verificar se o e-mail já existe.
10. Verificar se CPF/CNPJ já existe.
11. Comparar senha e confirmação.
12. Forçar perfil USER, impedindo cadastro público como ADMIN.

13. Gerar hash BCrypt e salvar Usuario.
14. Cadastrar cartão inicial tokenizado dentro da mesma transação.
15. Devolver UsuarioDTO sem segredos.

Por que @Transactional?

Se salvar o usuário funcionar, mas cadastrar o cartão falhar, a transação evita deixar um cadastro incompleto. A operação inteira é confirmada ou desfeita.

17. RF3 - Senha com hash BCrypt

O banco não guarda a senha digitada. UsuarioService chama passwordEncoder.encode e salva o resultado no campo senhaHash. O BCrypt inclui salt e gera hashes diferentes mesmo para senhas iguais.

```
usuario.setSenhaHash(passwordEncoder.encode(dto.getSenha()));
```

Durante o login, a comparação não é feita com equals. AuthService usa passwordEncoder.matches para comparar a senha digitada ao hash persistido.

```
boolean senhaConfere = passwordEncoder.matches(dto.getSenha(), usuario.getSenhaHash());
```

Frase pronta

A senha nunca é persistida em texto puro. No cadastro eu salvo o hash BCrypt em senhaHash e, no login, uso matches para validar a senha informada.

18. RF4 - API de login

A rota é POST /api/v1/auth/login. Ela recebe LoginRequestDTO e devolve LoginResponseDTO.

```
{
  "email": "maria@example.com",
  "senha": "Senha@123"
}
```

AuthService busca o usuário pelo e-mail. Se não achar ou a senha estiver errada, retorna 401. Se estiver certa, devolve id, nome, e-mail, perfil e mensagem de sucesso.

Atenção: login x rotas protegidas

O endpoint /auth/login comprova as credenciais, mas não emite JWT nem cria sessão. Para acessar rotas protegidas, o projeto usa HTTP Basic: o cliente envia usuário e senha a cada requisição.

19. RF7 - Alteração do cartão após autenticação

As rotas `/api/v1/metodos-pagamento/**` exigem autenticação. O PUT recebe `AtualizacaoCartaoDTO` e `Authentication`. O Spring Security informa o e-mail autenticado por `authentication.getName()`.

```
@PutMapping("/{id}")
public MetodoPagamentoDTO atualizar(
    @PathVariable Long id,
    @Valid @RequestBody AtualizacaoCartaoDTO dto,
    Authentication authentication) {
    return metodoPagamentoService.atualizar(
        id, dto, authentication.getName());
}
```

O service valida o dono do cartão e armazena somente dados reduzidos:

Recebido na entrada	Persistido no banco?	O que fica salvo
Número completo	Não	Somente os últimos 4 dígitos e bandeira identificada.
CVV	Não	Nunca é persistido.
Validade	Sim, parcialmente	Mês e ano.
Titular	Sim	Nome do portador.
Token	Gerado localmente	UUID no campo tokenGateway.

`MetodoPagamentoService.autorizarUsuario` permite o próprio usuário e também o administrador. Outro usuário autenticado recebe 403 Forbidden.

20. RF8 - Catálogo detalhado de filmes e séries

Conteúdo representa filmes e séries em uma tabela única. O campo tipo diferencia FILME e SERIE. Isso evita duplicar lógica e também facilita separar favoritos.

Campo RF8	Campo Java	Exemplo
Título	titulo	Coringa
Gênero	genero	Drama
Ano	ano	2019
Duração	duracaoMinutos	122
Relevância	relevancia	8.5
Sinopse	sinopse	A origem de um vilão conhecido...
Trailer	trailerUrl	https://youtu.be/...

Endpoints úteis:

- ☐ GET `/api/v1/conteudos` - lista tudo em ordem alfabética.
- ☐ GET `/api/v1/conteudos/{id}` - mostra todos os detalhes.
- ☐ GET `/api/v1/conteudos?tipo=FILME` - filtra filmes.
- ☐ GET `/api/v1/conteudos?tipo=SERIE` - filtra séries.
- ☐ GET `/api/v1/conteudos?genero=Drama` - filtra por gênero.

- ❑ GET /api/v1/conteudos?q=Dark - pesquisa no título.
- ❑ GET /api/v1/conteudos/top-relevancia - ordena por relevância.
- ❑ GET /api/v1/conteudos/lancados-apos?ano=2015 - filtra por ano.

21. RF10 e RF9 - Favoritos

RF10 exige listar filmes e séries separadamente. O projeto também implementa RF9, embora ele não seja obrigatório: é possível adicionar um favorito por usuariold e conteudold.

```
POST /api/v1/favoritos
{
  "usuarioId": 1,
  "conteudoId": 41
}
```

FavoritoService verifica três condições antes de salvar:

16. O usuário existe.
17. O conteúdo existe.
18. A mesma combinação usuário + conteúdo ainda não foi favoritada.

Para listar separadamente:

```
GET /api/v1/favoritos/usuario/{usuarioId}/filmes
GET /api/v1/favoritos/usuario/{usuarioId}/series
```

FavoritoRepository usa JPQL com JOIN entre Favorito e Conteudo e filtra UPPER(c.tipo). Assim, um endpoint devolve apenas FILME e o outro apenas SERIE.

22. RF11 - Administrador para cadastrar conteúdos

SecurityConfig protege POST, PUT e DELETE de /api/v1/conteudos com hasRole("ADMIN"). Consultas GET continuam públicas.

```
.requestMatchers(HttpMethod.POST,
  "/api/v1/conteudos", "/api/v1/conteudos/**")
.hasRole("ADMIN")
```

Credenciais locais de demonstração:

```
Username: admin
Password: admin123
```

OpenApiConfig registra o esquema basicAuth para o Swagger exibir o botão Authorize. Depois de autorizar, o Swagger envia o cabeçalho Authorization: Basic ... automaticamente.

Detalhe técnico importante

SecurityConfig cria um novo objeto User para o administrador a cada autenticação. Isso evita que o

apagamento interno de credenciais pelo Spring inutilize o admin após o primeiro uso.

23. RNF1 e RNF2

Requisito	Como está atendido	Como demonstrar
RNF1 - Git	Projeto versionado no repositório Projeto_TecBack.	Mostrar histórico e remote origin no GitHub.
RNF2 - README	README.md contém pré-requisitos, execução, H2, Swagger e exemplos.	Abrir README.md e executar o Maven Wrapper.

Parte IV - Tópicos cobrados na avaliação

24. Critérios descritos pelo professor

Critério	Peso	Onde aparece no projeto
Entregas na data certa	20%	Processo acadêmico; não depende do código.
Bean Validation	20%	DTOs, @Valid, @CpfCnpj, @SenhaForte e @Pattern.
Services e serviço externo	20%	Camada service e integração ViaCEP com OpenFeign.
Persistência	10%	JPA, Hibernate, repositories e H2 em arquivo.
Padrões de projeto	10%	Camadas, Repository, DTO, Builder, DI e handler global.
Git	10%	Controle de versão e GitHub.
Swagger	10%	SpringDoc OpenAPI, Swagger UI e basicAuth.

25. JPQL no ConteudoRepository

JPQL se parece com SQL, mas consulta classes e atributos Java em vez de nomes físicos de tabelas.

ConteudoRepository possui cinco consultas com @Query e um método derivado.

Consulta	Trecho principal	Uso
JPQL 1	SELECT c FROM Conteudo c ORDER BY c.titulo ASC	Lista alfabética.
JPQL 2	WHERE LOWER(c.genero) = LOWER(:genero)	Filtro sem diferenciar maiúsculas.
JPQL 3	ORDER BY c.relevancia DESC	Ranking de relevância.
JPQL 4	WHERE c.ano > :ano ORDER BY c.ano DESC	Lançados após um ano.
JPQL 5	LOWER(c.titulo) LIKE LOWER(CONCAT('%', :q, '%'))	Pesquisa por palavra no título.
Derived Query	findAllByTipoOrderByTituloAsc(tipo)	Filtro por FILME ou SERIE.

Frase pronta

Em JPQL eu uso o nome da entidade Conteudo e seus atributos Java. O Hibernate traduz para SQL e consulta a tabela física no H2.

26. Padrões e boas práticas presentes

Prática ou padrão	Exemplo no projeto	Benefício
Arquitetura em camadas	Controller -> Service -> Repository	Responsabilidades separadas.
Repository Pattern	Interfaces que estendem JpaRepository	Centraliza acesso a dados.
DTO Pattern	CadastroUsuarioDTO, ConteudoDTO	Controla contrato e evita exposição indevida.
Dependency Injection	@RequiredArgsConstructor com campos final	Baixo acoplamento e testabilidade.
Builder Pattern	Lombok @Builder	Criação legível de objetos.
Declarative HTTP Client	ViaCepClient com @FeignClient	Integração externa simples.
Global Exception Handler	@RestControllerAdvice	Erros consistentes.
Tokenização demonstrativa	UUID em tokenGateway	Não persiste número completo nem CVV.

27. Tratamento global de erros

GlobalExceptionHandler centraliza respostas de erro e usa ProblemDetail. Isso evita repetir tratamento em cada controller e mantém status corretos.

Exceção	Resposta	Exemplo
MethodArgumentNotValidException	400 + errors por campo	Senha fraca ou tipo STRING.
CustomBeanException	400	CEP inválido retornado pelo ViaCEP.
ResponseStatusException	Status original	401, 403, 404 ou 409 criados pelos services.
Exception genérica	500	Erro inesperado.

O Content-Type da resposta é application/problem+json. Essa estrutura segue o formato ProblemDetail do Spring para comunicar erros de forma padronizada.

28. Serviço externo ViaCEP

O professor pontua uso de service e serviço externo. O projeto demonstra isso no módulo Funcionario. Quando um funcionário é cadastrado com CEP, FuncionarioService remove caracteres, chama ViaCepClient e preenche logradouro, bairro, localidade e UF.

```
@FeignClient(name = "viaCepClient",
              url = "${viacep.url:https://viacep.com.br/ws}")
public interface ViaCepClient {
    @GetMapping("/{cep}/json/")
    ViaCepResponseDTO buscarPorCep(@PathVariable("cep") String cep);
}
```

Endpoint de demonstração:

```
POST /funcionarios
{
  "nome": "Samantha",
  "cargo": "Analista",
  "cep": "58000-000"
}
```

Contexto

Funcionario é um módulo didático complementar, separado do domínio principal de streaming. Ele existe para demonstrar integração externa com OpenFeign.

29. Swagger e OpenAPI

Swagger UI é a documentação interativa gerada pelo SpringDoc. Ele lista endpoints, mostra schemas JSON e permite executar requisições. OpenApiConfig adiciona basicAuth para que o botão Authorize apareça.

```
Swagger UI: http://localhost:8080/swagger-ui/index.html
OpenAPI JSON: http://localhost:8080/v3/api-docs
```

30. Testes automatizados e JaCoCo

Os testes servem como prova repetível de comportamento. A execução final deste guia passou com 55 testes, 0 falhas e 0 erros. O relatório JaCoCo foi gerado com cobertura total de 45% das instruções e 47% dos branches.

Tipo de teste	Arquivo	O que comprova
Integração	ObrigatoriosIntegrationTest	Fluxo obrigatório completo, HTTP Basic e respostas HTTP.
Repository	ConteudoRepositoryTest	Quatro consultas JPQL principais.
Service	UsuarioServiceTest	Hash, perfil USER e cartão inicial.
Service	MetodoPagamentoServiceTest	Tokenização e bloqueio de outro usuário.
Validation	CpfCnpjValidatorTest	CPF/CNPJ válidos e inválidos.
Validation	SenhaForteValidatorTest	Regras de senha forte.
Legado	FilmeControllerTest, FilmeServiceTest etc.	CRUD e estrutura didática original.

```
.\mvnw.cmd test

# relatório após os testes
target/site/jacoco/index.html
```

31. Bonificações e itens não obrigatórios

Item	Situação atual	Observação honesta para apresentar
RF2 - confirmação por e-mail	Não implementado	Não é obrigatório.
RF9 - adicionar favorito	Implementado	POST /api/v1/favoritos.
RFB1 - planos diferentes	Estrutura implementada	Plano e Assinatura existem com endpoints.
RFB2 - limitar instâncias simultâneas	Não implementado	Não é obrigatório.
Flyway	Dependência presente	Desativado no perfil dev atual; testes possuem migration.
Testcontainers	Dependências presentes	Não é o foco da demonstração atual.

Parte V - Demonstração prática

32. Preparar o ambiente

19. Abra um terminal na pasta Projeto Flix.
20. Execute `.\mvnw.cmd spring-boot:run`.
21. Aguarde o servidor iniciar na porta 8080.
22. Abra <http://localhost:8080/swagger-ui/index.html>.
23. Use Ctrl+F5 se o navegador estiver mostrando uma versão antiga.

Se aparecer um pop-up do navegador pedindo login

Clique em Cancelar. Esse pop-up costuma ser uma resposta 401. Para a demonstração normal, use o botão Authorize dentro do Swagger e informe as credenciais adequadas.

33. Roteiro recomendado no Swagger

O roteiro abaixo demonstra os principais requisitos em uma ordem fácil de explicar.

24. Cadastrar um usuário em POST `/api/v1/usuarios`.
25. Fazer login em POST `/api/v1/auth/login`.
26. Abrir Authorize e testar cartão com o e-mail e senha do usuário.
27. Atualizar o cartão em PUT `/api/v1/metodos-pagamento/{id}`.
28. Trocar Authorize para admin / admin123.
29. Cadastrar filme e série em POST `/api/v1/conteudos`.
30. Listar catálogo em GET `/api/v1/conteudos`.
31. Detalhar um conteúdo em GET `/api/v1/conteudos/{id}`.
32. Adicionar favorito em POST `/api/v1/favoritos`.
33. Listar favoritos em GET `.../filmes` e GET `.../series`.
34. Opcionalmente abrir o H2 e mostrar que os dados persistiram.

34. JSON para cadastro de usuário

Se Maria já estiver cadastrada no seu banco, use outro e-mail e outro CPF válido. O exemplo abaixo pode ser usado em um banco limpo.

```
{
  "nomeCompleto": "Ana Souza",
  "dataNascimento": "1998-08-15",
  "email": "ana@example.com",
  "senha": "Senha@123",
```

```

"confirmarSenha": "Senha@123",
"cpfCnpj": "11144477735",
"numeroCartao": "4111111111111111",
"validadeCartao": "12/2030",
"codigoSegurancaCartao": "123",
"nomeTitularCartao": "Ana Souza"
}

```

35. JSON para login

```

{
  "email": "maria@example.com",
  "senha": "Senha@123"
}

```

A Maria já existe no banco persistente verificado durante a revisão. Em um banco novo, faça login com o usuário que você acabou de cadastrar.

36. Atualização de cartão

No Swagger, clique em Authorize e informe o e-mail e a senha do dono do cartão. Depois abra PUT /api/v1/metodos-pagamento/{id}. Use o ID retornado ou conferido no GET de métodos do usuário.

```

{
  "numeroCartao": "555555555554444",
  "validadeCartao": "12/2031",
  "codigoSegurancaCartao": "321",
  "nomeTitularCartao": "Maria da Silva"
}

```

37. Cadastro de conteúdo como administrador

No Swagger, abra Authorize e substitua as credenciais pelo administrador local. Depois execute POST /api/v1/conteudos.

```

Username: admin
Password: admin123

```

```

{
  "titulo": "Coringa",
  "tipo": "FILME",
  "ano": 2019,
  "duracaoMinutos": 122,
  "relevancia": 8.5,
  "sinopse": "A origem de um dos vilões mais conhecidos dos quadrinhos.",
  "trailerUrl": "https://youtu.be/zAGVQLHvwOY",
  "genero": "Drama"
}

```

```

{
  "titulo": "Dark",
  "tipo": "SERIE",
  "ano": 2017,
  "duracaoMinutos": 60,
  "relevancia": 9.3,
  "sinopse": "Mistérios atravessam gerações em uma pequena cidade alemã.",
}

```



```
"trailerUrl": "https://example.com/dark",
"genero": "Ficção Científica"
}
```

38. Favoritos e conferência

Use os IDs realmente retornados pelo seu banco. Em um banco persistente, IDs podem não começar em 1 porque tentativas anteriores continuam registradas.

```
POST /api/v1/favoritos
{
  "usuarioId": 1,
  "conteudoId": 41
}

GET /api/v1/conteudos
GET /api/v1/favoritos/usuario/1/filmes
GET /api/v1/favoritos/usuario/1/series
```

Se a resposta de séries for [], a API está funcionando: significa apenas que aquele usuário ainda não favoritou uma série.

39. Conferir dados pelo H2

Abra <http://localhost:8080/h2> e conecte usando a configuração do perfil dev.

```
SELECT * FROM USUARIOS;
SELECT * FROM METODO_PAGAMENTO;
SELECT * FROM CONTEUDO ORDER BY TITULO;
SELECT * FROM FAVORITOS;
SELECT * FROM PLANO;
SELECT * FROM ASSINATURA;
```

O que observar no cartão

A tabela METODO_PAGAMENTO não deve possuir número completo do cartão nem CVV. Confira ULTIMOS4, BANDEIRA, MES_EXP, ANO_EXP, NOME_PORTADOR e TOKEN_GATEWAY.

40. Estado do banco verificado em 01/06/2026

Durante a revisão final, o H2 persistente continha registros válidos para demonstração e também alguns dados antigos genéricos. Eles não impedem o funcionamento, mas vale limpar antes de apresentar.

Situação observada	Ação recomendada
Coringa, Dark e Breaking Bad cadastrados	Manter para demonstração.
Interestelar duplicado	Apagar uma cópia se quiser deixar a lista limpa.
Dois conteúdos antigos titulo=string e tipo=STRING	Apagar antes da apresentação.
Usuário 1 possui filme favorito, mas nenhuma série	Favoritar Dark para demonstrar as duas listas.

Importante

A API agora recusa novos tipos STRING com HTTP 400. Os registros genéricos já existentes são anteriores à validação e só permanecem porque o H2 é persistente.

Parte VI - Preparação para apresentação

41. Roteiro oral pronto

Você pode adaptar este texto à sua forma de falar:

Abertura sugerida

Meu projeto é uma API REST chamada IESPFLIX, feita com Java 17 e Spring Boot. Ela simula o backend de uma plataforma de streaming. O código foi organizado em camadas: Controller recebe as requisições, Service aplica regras de negócio, Repository acessa o banco H2, Model representa as tabelas e DTO controla os dados de entrada e saída.

- ☐ No RF1, o cadastro recebe dados pessoais, confirmação de senha e cartão. Eu salvo o usuário e tokenizo o cartão na mesma transação.
- ☐ No RF3, a senha não vai para o banco em texto puro: eu salvo um hash BCrypt em senhaHash.
- ☐ No RF4, o endpoint de login busca o usuário por e-mail e usa BCrypt matches.
- ☐ No RF7, a rota de cartão exige HTTP Basic e confere se o cartão pertence ao usuário autenticado.
- ☐ No RF8, a entidade Conteudo representa filmes e séries com título, gênero, ano, duração, relevância, sinopse e trailer.
- ☐ No RF10, FavoritoRepository faz JOIN com Conteudo e lista FILME e SERIE separadamente.
- ☐ No RF11, SecurityConfig exige ROLE_ADMIN para criar, alterar ou excluir conteúdos.
- ☐ Também uso Bean Validation, handler global de erros, Swagger, H2 persistente, JPQL, Git, README e integração ViaCEP com OpenFeign.

42. Perguntas prováveis e respostas

Pergunta	Resposta curta
Por que usar DTO?	Para controlar o contrato da API, validar entradas e evitar expor campos sensíveis.
Por que não salvar a senha?	Salvo hash BCrypt. No login, uso matches para comparar.
Login gera token JWT?	Não. O endpoint valida credenciais; rotas protegidas usam HTTP Basic.
Onde está RF7?	PUT /api/v1/metodos-pagamento/{id}, com authentication.getName e conferência do dono.
Por que não salvar CVV?	É dado sensível e desnecessário após validação. O projeto nunca persiste CVV.
Como filmes e séries são separados?	Conteudo.tipo recebe FILME ou SERIE e FavoritoRepository filtra esse campo.
O que é JPQL?	Consulta entidades e atributos Java; Hibernate converte para SQL.

Pergunta	Resposta curta
O que é Repository?	Interface Spring Data que abstrai acesso ao banco.
O que faz @Valid?	Ativa Bean Validation antes de executar o service.
Onde está o serviço externo?	ViaCepClient com @FeignClient, chamado por FuncionarioService.
Por que H2 continua com dados?	O perfil dev usa jdbc:h2:file e ddl-auto=update.
Qual a diferença entre 401 e 403?	401: falta autenticação válida. 403: autenticado sem permissão.
Por que existe Filme e Conteudo?	Filme é módulo didático antigo. Conteudo é o catálogo principal do IESPFlix.

43. Checklist antes de apresentar

- ☐ ☐ Executar .\mvnw.cmd test e confirmar 55 testes sem falhas.
- ☐ ☐ Iniciar a aplicação e abrir Swagger UI.
- ☐ ☐ Limpar conteúdos antigos titulo=string e duplicidade de Interestelar no H2.
- ☐ ☐ Cadastrar ou manter pelo menos dois filmes e duas séries.
- ☐ ☐ Garantir que o usuário de demonstração possui ao menos um filme e uma série favoritos.
- ☐ ☐ Testar POST /api/v1/auth/login.
- ☐ ☐ Testar Authorize com admin / admin123.
- ☐ ☐ Cadastrar conteúdo como admin e confirmar 201 Created.
- ☐ ☐ Tentar cadastrar conteúdo sem admin e explicar 401 ou 403.
- ☐ ☐ Abrir H2 e mostrar que senha é hash e cartão não contém número completo nem CVV.
- ☐ ☐ Mostrar README.md.
- ☐ ☐ Fazer commit e push finais no GitHub.

44. Glossário rápido

Termo	Definição
API REST	Interface HTTP organizada por recursos, verbos e status codes.
Endpoint	URL e verbo que executam uma operação.
JSON	Formato textual de entrada e saída da API.
Spring Boot	Framework que configura e executa a aplicação Java.
Bean	Objeto criado e gerenciado pelo Spring.
Injeção de dependência	Spring entrega objetos necessários ao construtor da classe.
Entity	Classe Java persistida como tabela.
DTO	Objeto usado para transportar dados pela API.
Repository	Camada de acesso ao banco.
Service	Camada com regras de negócio.
JPA	Especificação Java para persistência.
Hibernate	Implementação ORM usada para traduzir objetos e SQL.
JPQL	Linguagem de consulta de entidades Java.
H2	Banco leve usado localmente.
BCrypt	Algoritmo adequado para hash de senhas.
HTTP Basic	Autenticação que envia credenciais em cada requisição.

Termo	Definição
Swagger / OpenAPI	Documentação interativa da API.
OpenFeign	Cliente HTTP declarativo para APIs externas.
JaCoCo	Ferramenta de cobertura de testes.

45. Resumo de bolso

Cinco ideias para memorizar

1. Controller recebe HTTP. 2. DTO valida e protege dados. 3. Service aplica regra de negócio. 4. Repository conversa com H2. 5. SecurityConfig separa rotas públicas, autenticadas e administrativas.

Você não precisa decorar cada linha. O objetivo é conseguir explicar o caminho da informação, mostrar onde cada requisito está implementado e demonstrar o comportamento no Swagger com calma.

Fim do guia.